

Competitive Programming

Section 30: GCD, LCM, and Diophantine Equations

Competitive Programming Course

2026-07-06

Table of contents

1	Why gcd is a contest primitive	3
2	Euclid's algorithm	4
3	LCM	5
4	Extended Euclid	6
5	Modular inverse with any modulus	7
6	Linear Diophantine equations	8
7	Worked example	9
8	C++ notes	10
9	Python notes	11
10	Common mistakes	12
11	Submission checklist	13

1 Why gcd is a contest primitive

The greatest common divisor answers the question:

What is the largest unit size that divides both numbers?

It appears in fraction reduction, divisibility checks, modular inverses, periods, grid-step problems, and equations with integer solutions.

2 Euclid's algorithm

The key identity:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

Why? Any number that divides both a and b also divides $a - q*b$, including the remainder $a \bmod b$. The common divisors are unchanged.

Algorithm:

```
while b != 0:
    a, b = b, a mod b
answer = a
```

Complexity is $O(\log \min(a, b))$.

3 LCM

The least common multiple satisfies:

$$\text{gcd}(a,b) * \text{lcm}(a,b) = a * b$$

Compute it as:

$$\text{lcm} = a / \text{gcd}(a,b) * b$$

Divide before multiplying to reduce overflow risk.

4 Extended Euclid

Extended Euclid finds integers x and y such that:

$$a*x + b*y = \gcd(a,b)$$

This is called Bezout's identity.

Recursive transition:

```
If b == 0:
    gcd = a, x = 1, y = 0

Otherwise:
    g, x1, y1 solve b*x1 + (a mod b)*y1 = g
```

Since:

$$a \bmod b = a - \text{floor}(a/b)*b$$

Substitute:

$$\begin{aligned} b*x1 + (a - \text{floor}(a/b)*b)*y1 \\ = a*y1 + b*(x1 - \text{floor}(a/b)*y1) \end{aligned}$$

So:

$$\begin{aligned} x &= y1 \\ y &= x1 - \text{floor}(a/b)*y1 \end{aligned}$$

5 Modular inverse with any modulus

An inverse of a modulo m is an x such that:

$$a*x \equiv 1 \pmod{m}$$

This means:

$$a*x + m*y = 1$$

Such integers exist exactly when:

$$\gcd(a, m) = 1$$

So extended Euclid gives the inverse directly. This is more general than the Fermat inverse from Section 29, which required a prime modulus.

6 Linear Diophantine equations

For:

$$a*x + b*y = c$$

integer solutions exist exactly when:

$$\gcd(a,b) \text{ divides } c$$

If extended Euclid gives:

$$a*x_0 + b*y_0 = g$$

and $c = g * \text{scale}$, then:

$$a*(x_0*\text{scale}) + b*(y_0*\text{scale}) = c$$

This gives one solution. There are infinitely many others when a solution exists, but the local problem only asks for any one.

7 Worked example

Solve:

$$6x + 10y = 14$$

$\gcd(6, 10) = 2$, and 2 divides 14, so a solution exists.

Extended Euclid may produce:

$$6 \cdot 2 + 10 \cdot (-1) = 2$$

Scale by 7:

$$\begin{aligned} x &= 14 \\ y &= -7 \end{aligned}$$

Check:

$$6 \cdot 14 + 10 \cdot (-7) = 84 - 70 = 14$$

8 C++ notes

C++ has `std::gcd` in `<numeric>`, included by `bits/stdc++.h`.

Normalize modular inverse answers:

```
x %= m;  
if (x < 0) x += m;
```

For the local constraints, `long long` is enough. For larger `1e18` versions, extended-Euclid coefficient multiplication may require `__int128`.

9 Python notes

Python has:

```
math.gcd(a, b)
```

Python's % already normalizes negatives:

```
x % m
```

is in $[0, m-1]$ when $m > 0$.

10 Common mistakes

```
Computing lcm as a*b/g before dividing.  
Assuming every number has a modular inverse.  
Using Fermat inverse with composite modulus.  
Forgetting to check  $c \% \gcd(a,b)$  for  $ax + by = c$ .  
Expecting the Diophantine solution to be unique.  
Failing to normalize negative inverse coefficients in C++.
```

11 Submission checklist

1. Did you compute gcd first?
2. For lcm, did you divide before multiplying?
3. For inverse, is $\gcd(a,m)$ exactly 1?
4. For $ax+by=c$, does $\gcd(a,b)$ divide c ?
5. Are negative modular answers normalized?